

Progetto BEM: metodo di collocazione
per lo scattering di onde su uno o più
ostacoli poligonali. Scattering di
ostacoli curvilinei e estensione al
metodo di Galerkin.

Metodi numerici avanzati per le PDEs
Schipor Elena Ionela, mat 565930, 3/10/2026

1 Metodo di collocazione per scattering di onde su un ostacolo

1.1 Notazione e definizione del problema

Consideriamo il seguente setting: sia u_{inc} una soluzione qualsiasi dell'equazione di Helmholtz, che chiameremo **onda incidente**. Sia Ω un ostacolo chiuso, detto anche scatterer, che possiamo rappresentare tramite un aperto Lipschitziano di $\mathbb{R}^2$. Più precisamente, chiameremo Ω^- la parte interna dell'ostacolo, che l'onda non attraversa, mentre Ω^+ è la parte esterna che a noi interessa. inoltre, sia $\Gamma = \partial\Omega$, che ha un orientamento con normale $\vec{n}$ uscente da Ω^- . Definiamo il problema di diffusione (scattering) con condizione al bordo sound-soft (**SSSP- sound soft scattering problem**) come segue: cerchiamo $u_{scat} \in H_{loc}^1(\Omega^+)$ tale che

$$\begin{aligned} \Delta u_{scat} + k^2 u_{scat} &= 0 \quad in \Omega; \\ \gamma^+(u_{scat} + u_{inc}) &= 0 \quad in \Gamma; \\ u_{scat} &\text{ è uscente.} \end{aligned} \tag{1}$$

In particolare, ricordiamo che una funzione (un'onda) u è uscente, o soddisfa la condizione di Sommerfeld, quando:

$$|\partial_r u - iku| = o(r^{-1/2}) \quad per \ r \rightarrow \infty.$$

Questa condizione, in pratica, impone che le soluzioni siano onde che “decadono” man mano che ci si allontana da Ω . Inoltre, $\gamma^+ : H_{loc}^1(\Omega) \rightarrow H^{\frac{1}{2}}(\partial\Omega)$ è l'operatore di traccia seguendo l'orientamento di Γ .

Questo problema può essere ricondotto al problema più generale chiamato problema esterno di Dirichlet (EDP), in cui dato $g_D \in H^{\frac{1}{2}}(\Gamma)$, cerchiamo $u \in H_{loc}^1(\Omega)$ tale che:

$$\begin{aligned} \Delta u + k^2 u &= 0 \quad in \Omega; \\ \gamma^+ u &= g_D \quad in \Gamma; \\ u &\text{ è uscente.} \end{aligned} \tag{2}$$

Definiamo anche la soluzione fondamentale dell'equazione di Helmholtz:

$$\Phi_k(x, y) = \frac{i}{4} H_0^{(1)}(k|x - y|),$$

con $x \neq y \in \mathbb{R}^2$. Definiamo anche il single-layer potential $\mathcal{S}$:

$$(\mathcal{S}\psi)(x) := \int_{\Gamma} \Phi_k(x, y) \psi(y) ds(y), \quad x \in \Omega_+,$$

e il single layer operator S :

$$(S\psi)(x) := \int_{\Gamma} \Phi_k(x, y) \psi(y) ds(y), \quad x \in \Gamma.$$

Osserviamo che questi due operatori sono praticamente uguali, tranne per il dominio di definizione. Inoltre, a causa della presenza delle funzioni di Hankel $H_0^{(1)}$, gli integrali nel single layer operator S possono essere singolari.

Questa funzione e questi operatori saranno fondamentali per la costruzione dei metodi numerici per la risoluzione di (1).

Durante il corso, abbiamo già studiato alcune proprietà di questi operatori, che permettono di dire che il problema (2) è equivalente a cercare $\psi \in H^{-\frac{1}{2}}(\Gamma)$ tale che:

$$\begin{aligned} S\psi &= g_D \quad \text{in } \Gamma; \\ \implies u &= \mathcal{S}\psi \quad \text{in } \Omega_+ \end{aligned} \tag{3}$$

La prima equazione di (3) appartiene alla classe delle equazioni integrali al bordo (BIE), mentre la seconda è la formula di rappresentazione che permette di ricavare u .

1.2 Metodo di Collocazione per BIE

A questo punto possiamo definire il metodo di collocazione per risolvere (SSSP) per un ostacolo poligonale. Il metodo degli elementi al bordo (BEM), nel caso del metodo di collocazione, inizia scegliendo N punti $x_1 \dots x_N$ su Γ , e fissando uno spazio discreto di funzioni V_N . Cerchiamo $\psi_N \in V_N$ tale che $(S\psi_N)(x_j) = g_D(x_j)$ per ogni j da 1 a N .

Possiamo suddividere Γ poligonale in più elementi rettilinei che chiameremo K_j , in modo che $\bigcup_{j=1}^N K_j = \Gamma$. Chiameremo p_j, p_{j+1} gli estremi di K_j , con $h = |p_j - p_{j+1}|$ e $x_j = (p_j + p_{j+1})/2$. Chiamiamo anche τ_j la direzione del segmento K_j , ovvero $\frac{p_{j+1} - p_j}{|p_{j+1} - p_j|}$.

Come spazio di funzioni, prendiamo per ora delle funzioni costante a tratti. In particolare, una scelta semplice di base per questo spazio è $\phi_j = \mathbb{1}_{K_j}$. In questo modo, possiamo scrivere la soluzione cercata $\psi_N = \sum_{j=1}^N \Psi_j \phi_j$. Noi dobbiamo risolvere $S\psi_N = g_D$, ma se lavoriamo puntualmente otteniamo un sistema lineare $A_{col} \Psi = F_{col}$ dove:

$$A_{col}^{j,m} := S\phi_m(x_j) = \int_{\Gamma} \Phi_k(x_j, y) \phi_m(y) ds(y), \quad F_{col}^j := g_D(x_j). \tag{4}$$

Per poter estendere questo metodo allo scattering con più oggetti, sarà necessario fare attenzione alla costruzione degli elementi. L'implementazione

di questo metodo parte dalla costruzione della mesh su Γ , usando delle funzioni opportune.

La prima cosa che abbiamo a disposizione è un cell array `V_collection`, che contiene al suo interno uno o più array che contengono a loro volta i vertici dei poligoni che definiscono lo scatterer. Inoltre, fissiamo il valore di k nell'equazione di Helmholtz, che ci permette di trovare una taglia della mesh `h_ideale` che dipende da k , in modo da non ignorare le oscillazioni della soluzione. Useremo i numeri complessi con la convenzione $x + iy \longleftrightarrow (x, y)$.

```

1 V = \{[0, 1, 1i]\}; % caso con un solo scatterer
2 k = 20;
3 h_ideale = pi/(10*k);

```

Con questi elementi possiamo costruire la nostra mesh; cominciamo calcolando il numero di punti che servono per ogni lato del poligono, con la funzione `find_ideal_mesh`. All'interno di essa viene anche richiamata la funzione `lati_dir(V)`, che a un array `V` associa le lunghezze dei lati e i vettori direzione complessivi.

```

1 function Ns = find_ideal_mesh(V, h_ideale)
2     [lati, ~] = lati_dir(V);
3     Ns = zeros(1, length(V));
4     for i = 1:length(V)
5         Ns(i) = ceil(lati(i)/h_ideale);
6     end
7 end

```

In particolare, il vettore `Ns` contiene tanti valori quanti i vertici (o i lati) del poligono, e ogni entrata è una stima di quanti punti servono per lato affinché la taglia della mesh sia vicina a `h_ideale`. Ora possiamo costruire la mesh sul nostro scatterer con la funzione `create_nodes`:

```

1 function [Nodes, Midpoints, Hj, Dirs] = create_nodes(
2     V_collection, h_ideale)
3     % input: V_collection, i vertici dei poligoni
4     % h, la taglia della mesh
5
6     Ns_totali = cell(1, length(V_collection));
7     N = 0; % numero di nodi totali
8     ind = 1;
9     for V_ = V_collection % per ogni poligono...
10        V = V_ \{: \}; % estraggo un poligono dalla cell
11        Ns = find_ideal_mesh(V, h_ideale);
12        Ns_totali(ind) = \{Ns\};
13        ind = ind + 1;
14        N = N + sum(Ns);
15    end
16    % inizializzo

```

```

16 Nodes = zeros(1,N); % p_j
17 Midpoints = Nodes; % x_j
18 Dirs = Nodes; % tau_j
19 Hj = Nodes; %h_j, nel caso in cui cambiassero tra un lato
    e l'altro
20 ind = 1;
21 idx = 1;
22 for V_ = V_collection
23     V = V_ \{: \};
24     Ns = Ns_totali \{ind \};
25     [lati, direzioni] = lati_dir(V);
26     % primo nodo = primo vertice
27     Nodes(idx) = V(1);
28     for j = 1:length(lati)
29         N = Ns(j);
30         h = lati(j)/N; % h puo' variare tra lati diversi
            ma non sullo stesso lato
31         tau = direzioni(j);
32         for i = 0:(N-1)
33             Nodes(idx) = V(j) + h*tau*i;
34             Midpoints(idx) = Nodes(idx) + h/2*tau;
35             Hj(idx) = h;
36             Dirs(idx) = tau;
37             idx = idx+1;
38         end
39     end
40     ind = ind+1;
41 end
42 end

```

Osserviamo che in generale, ogni elemento H_j può essere parametrizzato come segue, tramite una funzione di questa forma:

$$\mathbb{X}_j : (0, h_j) \rightarrow K_j, \quad \mathbb{X}_j(s) = p_j + s \cdot \tau_j. \quad (5)$$

In questo modo, la matrice di collocazione diventa:

$$\begin{aligned} A_{col}^{j,m} &= \int_{H_m} \Phi_k(x_j, y) ds(y) \\ \implies A_{col}^{j,m} &= \int_0^{h_m} H_0^{(1)}(k|p_m + s\tau_m - x_j|) ds. \end{aligned} \quad (6)$$

Dei semplici calcoli manuali, insieme all'espansione di $H_0^{(1)}$:

$$H_0^{(1)}(z) \approx 1 + \frac{2}{\pi} (\ln(\frac{z}{2} + \gamma)) \quad (7)$$

per $z \rightarrow 0$, dove γ è una costante [NIS], permettono di dire che l'integrale singolare in $A_{col}^{j,j}$ è:

$$A_{col}^{j,j} \approx \left(\frac{i}{2} - \frac{1}{\pi}\gamma + \frac{1}{\pi}\ln(2)\right)\frac{h_j}{2} - \frac{1}{\pi}\frac{h_j}{2}\ln(k\frac{h_j}{2}) + \frac{1}{\pi}\frac{h_j}{2}.$$

In questo modo, dopo aver applicato una quadratura opportuna, possiamo costruire la matrice il vettore con la funzione `build_collocation`:

```

1 function [A, F] = build_collocation(V, h_ideale, k,
    dato_bordo, q_quadratura)
2 %% V = vertici del poligono
3 % h_ideale = taglia (circa) della mesh
4 % k = k della eq di Helmholtz
5 % dato_bordo = dato al bordo
6 [Nodes, Midpoints, Hj, Dirs] = create_nodes(V, h_ideale);
7 N = length(Nodes);
8 A = zeros(N);
9 [nodi_quadratura, w] = gaussquad(q_quadratura);
10 F = zeros(N,1);
11 for j=1:N
12     xj = Midpoints(j);
13     for m=1:N
14         if j ~= m % caso NON singolare
15             hm = Hj(m);
16             pm = Nodes(m);
17             tau_m = Dirs(m);
18             A(j,m) = 0.25i*hm*sum(w.*besselh(0, k*abs(pm + (
                hm*nodi_quadratura )*tau_m - xj))));
19         else % CASO SINGOLARE
20             hj = Hj(j);
21             eugamma = -psi(1);
22             A(j,j) = (0.5i - (1/pi)*eugamma + (1/pi)*log(2))*
                hj/2 - (1/pi)*(hj/2)*log(k*hj/2) + (1/pi)*hj
                /2;
23         end
24     end
25     F(j) = dato_bordo(xj);
26 end
27 end

```

Una volta costruite la matrice A_{col} e il vettore F_{col} , non ci resta che risolvere il sistema $A_{col}\Psi = F_{col}$ e usare la formula di rappresentazione in (3):

```

1 function u = build_2D_solution(V, h_ideale, k, onda_incidente
    ,a,b,n_points, q_quadratura)
2 %% onda incidente: funzione del tipo @(x) ...
3 % a, b sono gli estremi del quadrato dove valuto la soluzione
4 % n_points il numero di punti su cui valuto la soluzione
5 [Nodes, Midpoints, Hj, Dirs] = create_nodes(V, h_ideale);

```

```

6  gD = @(x) - onda_incidente(x); % - traccia di onda incidente
   (ci riconduciamo al (EDP))
7  [A, F] = build_collocation(V, h_ideale, k, gD, q_quadratura);
8  PSI = A\F; % soluzione sul bordo
9  x = linspace(a,b,n_points);
10 [xx,yy] = meshgrid(x,x);
11 zz = xx + 1i * yy;
12 [s,w] = gaussquad(q_quadratura);
13
14 % applico formula di rappresentazione
15 Z = reshape(zz, [], 1);
16 Y = Nodes-Dirs.*Hj.*s;
17 Y = reshape(Y, 1, size(Y,1), size(Y,2));
18 dist = abs(Z-Y); % vettore |x-y| che va in input a besselh
19 G = besselh(0,1, k*dist);
20
21 SLP_VALS= G.*transpose(w);
22 c =sum(SLP_VALS, 2); % somma per discretizzare integrale
23 c = reshape(c, n_points^2, []);
24 P = Hj .* c;
25
26 u = 0.25i * sum(transpose(PSI).*P, 2); % somma su M
27 u = reshape(u, n_points, n_points);
28 end

```

A questo punto, abbiamo costruito tutte le funzioni che ci servono per cercare le soluzioni del problema di scattering contro uno o più poligoni, con qualsiasi onda incidente. Un esempio di test potrebbe essere il seguente:

```

1  a = -1;
2  b = 2;
3  n_points = 200;
4  V = {[0, 1, 1i]};
5  k = 20;
6  h_ideale = pi/(10*k); % per tenere sotto controllo le
   oscillazioni
7  q_quadratura = ceil(3*k); % anche qui
8  [Nodes, Midpoints, Hj, Dirs] = create_nodes(V, h_ideale);
9  % onda incidente: onda piana con direzione d
10 d = [1/2, sqrt(3)/2];
11 plane_wave = @(z) cos(k*(real(z)*d(1)+imag(z)*d(2))) + 1i*sin
   (k*(real(z)*d(1)+imag(z)*d(2)));
12 onda_incidente = plane_wave;
13 u =build_2D_solution(V, h_ideale,k, onda_incidente,a,b,
   n_points, q_quadratura);
14 x = linspace(a,b,n_points);
15 [xx,yy] = meshgrid(x,x);
16 zz = xx + 1i * yy;
17 u_inc = arrayfun(@(z) onda_incidente(z), zz) ;

```

```
18 | u_tot = u + u_inc;
```

Se facciamo i grafici di quanto calcolato, l'output è in figura 1. Possiamo anche studiare altre situazioni, per esempio in cui l'onda incidente è la soluzione fondamentale centrata in un punto x_0 fissato, ottenendo l'output in figura 2.

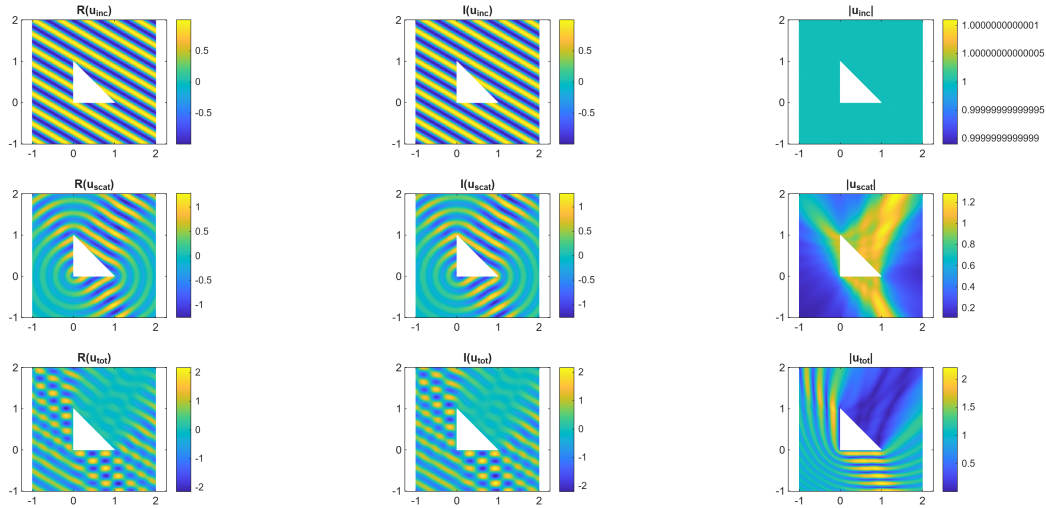


Figure 1: Parte reale, parte immaginaria e valore assoluto dell'onda incidente, onda diffusa e onda totale, nel caso in cui $k = 20$ e l'onda incidente è un'onda piana con direzione $d = [\frac{1}{2}, \frac{\sqrt{3}}{2}]$.

Possiamo anche testare lo scattering di più poligoni, per esempio due triangoli, come in figura 3.

Lo stesso codice può essere adeguato al caso degli ostacoli curvilinei, con il seguente accorgimento. Se un ostacolo curvilineo ha un'equazione parametrica del tipo $(x(t), y(t))$, basterà calcolare questi valori in una mesh molto fine, per esempio:

```
1 | nr_punti = 200; % sampling piu' fine piu' vicino alla forma
   | dell'ostacolo
2 |
3 | t = linspace(0, 2*pi, nr_punti);
4 | % circonferenza
5 | x = cos(t)*(0.25); y = sin(t)*(0.25);
6 |
7 | % usiamo polyshape per creare il 'poligono' (ovvero la figura
   | discretizzata)
```

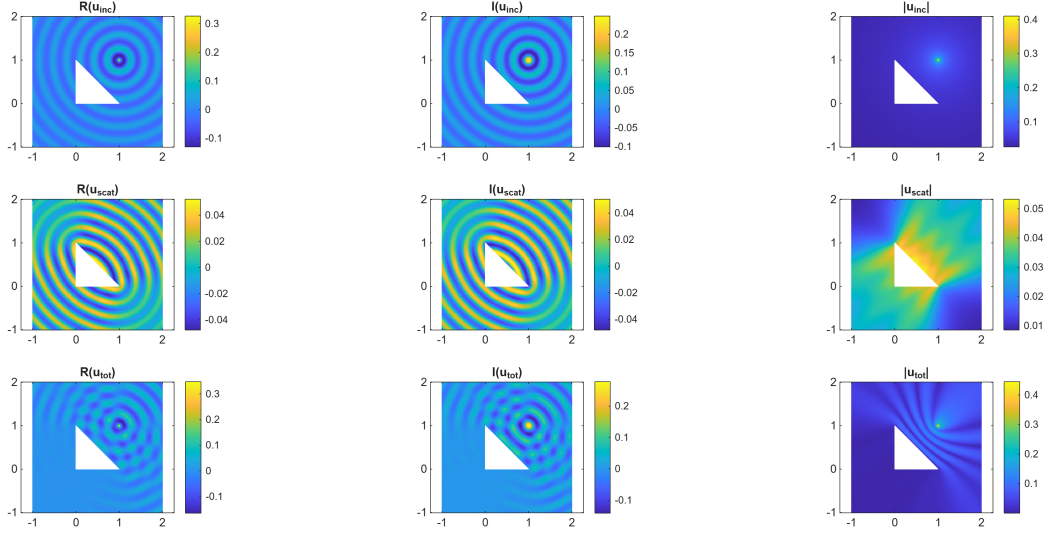


Figure 2: Parte reale, parte immaginaria e valore assoluto dell'onda incidente, onda diffusa e onda totale, nel caso in cui $k = 20$ e l'onda incidente è la soluzione fondamentale con origine $x_0 = [1, 1]$.

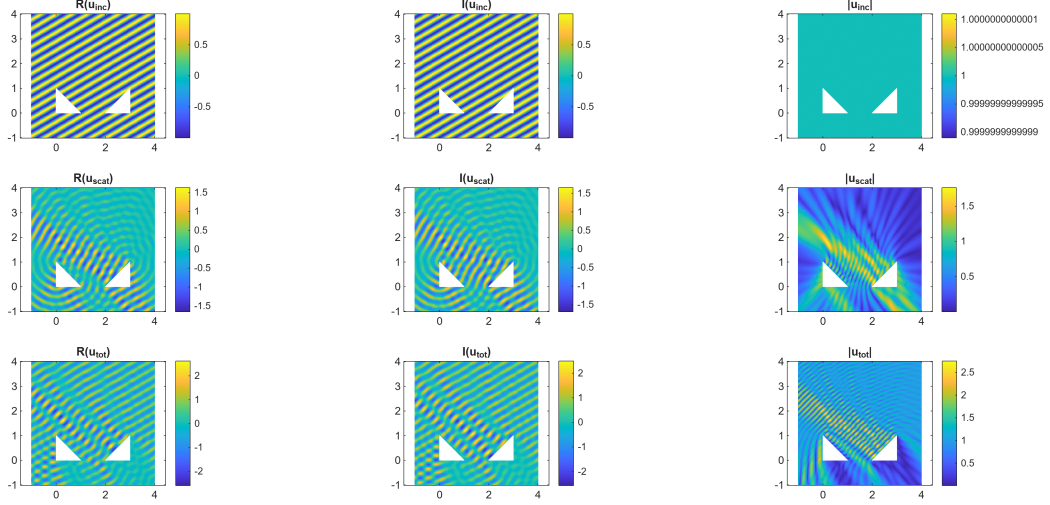


Figure 3: Caso con più poligoni, $k = 20$ e onda incidente piana con direzione $d = [\frac{1}{2}, -\frac{\sqrt{3}}{2}]$.

```

8  p = polyshape(x(1:end-1),y(1:end-1), '
   SolidBoundaryOrientation','ccw');
9  verts = p.Vertices;
10 V = {verts(:,1) + 1i*verts(:,2)}; % e ora lo passiamo al
   nostro solutore

```

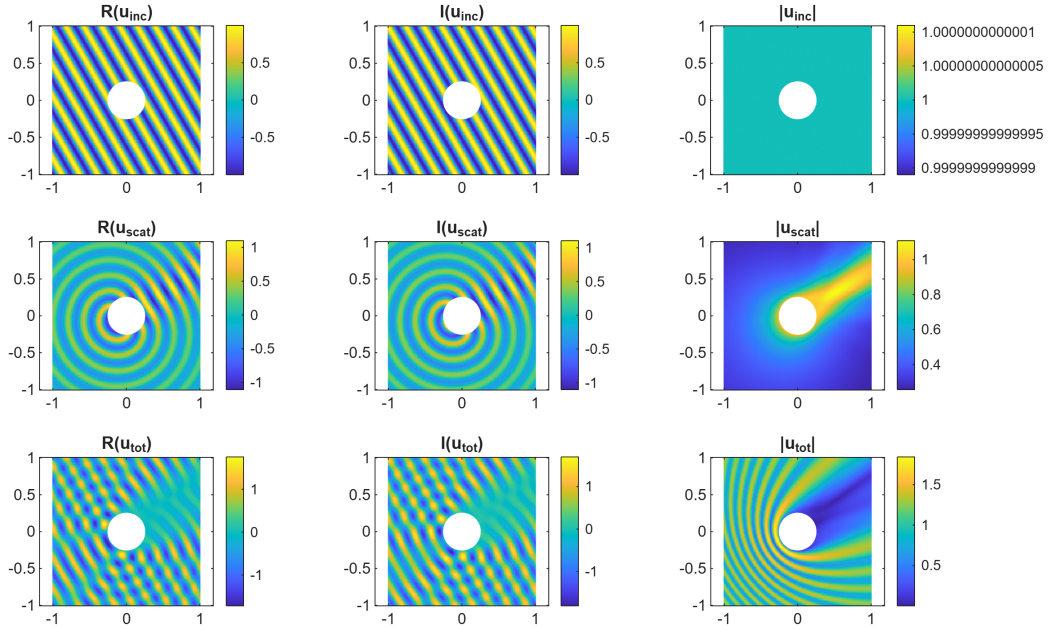


Figure 4: Ostacolo circolare con metodo di collocazione, u_{inc} = onda piana con $d = [\frac{\sqrt{3}}{2}, \frac{1}{2}]$, $k = 30$.

Possiamo confrontare i risultati sulla circonferenza con quelli dati dall'espansione con funzioni di Bessel nella sezione 4.3.1 delle dispense del corso. Qualitativamente, i plot in figura 4 sono simili a quelli visti durante il corso, ma possiamo confrontare i due metodi in modo quantitativo.

Infatti, ci basta riprodurre lo sviluppo ai multipoli per lo scattering di un disco circolare e vedere la differenza tra questa soluzione e la nostra soluzione proposta, per esempio in funzione del parametro `nr_punti`. Ci aspettiamo che all'aumentare del numero di vertici usati per approssimare lo scatterer liscio, la differenza tra u_{scatt} (ottenuta con il metodo di collocazione) e $u_{multipoli}$ (ottenuta con l'espansione ai multipoli) diminuisca.

Per trovare $u_{multipoli}$, in particolare, seguendo le indicazioni delle dispense del corso otteniamo il codice seguente:

```

1 a = -1;
2 b = 1;
3 n_points = 150;
4 k = 30;
5 x = linspace(a,b,n_points);
6 [xx,yy] = meshgrid(x,x);
7 zz = xx + 1i * yy;
8 L = 50; % serie in Z la approssimiamo come serie da -L a L
9 phi = pi/6;

```

```

10 rr = abs(zz);
11 theta = angle(zz); % in radianti
12
13 l = -L:L;
14 R = 0.25; % raggio circonferenza
15
16 J = arrayfun(@(x) besselj(x,k*R), l, 'UniformOutput',false);
17 H = arrayfun(@(x) besselh(x, k*rr)./besselh(x, k*R), l, '
    UniformOutput',false);
18
19 u_scatt_multipole = zeros(size(zz));
20 for j = 1:length(l)
21     a_l = 1i.^l(j) * exp(-1i*l(j)*phi)*cell2mat(J(j));
22     u_scatt_multipole = u_scatt_multipole - a_l.*cell2mat(H(j))
        .*exp(1i*l(j)*theta);
23 end

```

Confrontando questo output con quelli precedenti, per diversi valori di `nr_punti`, otteniamo il grafico in figura 5.

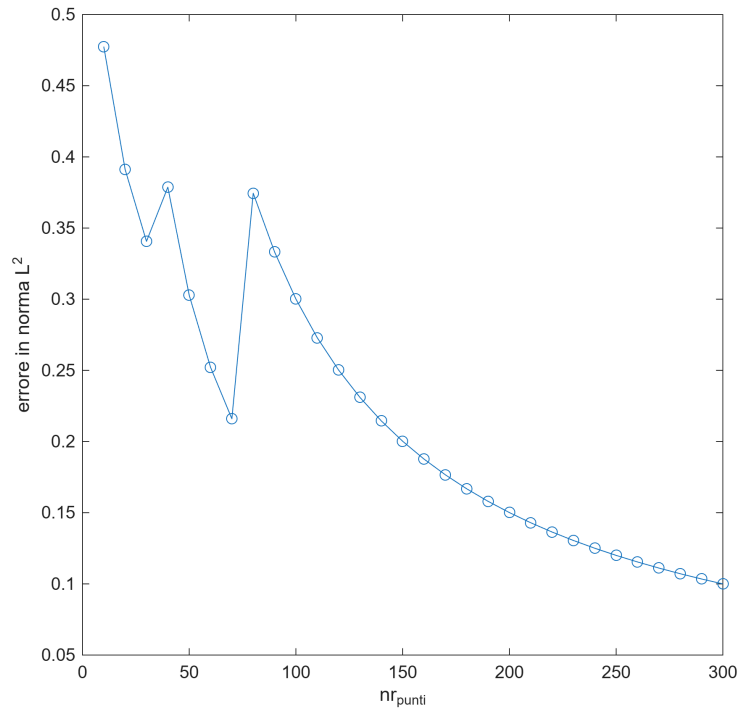


Figure 5: Andamento dell'errore relativo in norma $L^2(\Omega^+)$ della differenza tra u_{scatt} e $u_{multipoli}$, nel caso dello scattering contro un disco di raggio $R = 1/4$ e con $k = 30$. Sull'asse x ci sono il numero di punti usati per approssimare il disco come un poligono.

1.3 Metodo di Galerkin e confronto tra i due metodi

Ricordiamo che nel metodo di Galerkin per il BEM, siamo nello stesso contesto del metodo di collocazione precedentemente introdotto, ma invece di cercare $\psi_N \in V_N$ tale che $S\psi_N(x_j) = g_D(x_j)$ per ogni j , cerchiamo $\psi_N \in V_N$ tale che:

$$\mathcal{A}(\psi_N, \xi_N) = \int_{\Gamma} (S\psi_N) \overline{\xi_N} ds = \int g_D \overline{\xi_N} ds = F(\xi_N).$$

Come nel caso del metodo di collocazione, possiamo far corrispondere a questi operatori (in spazi di funzioni) delle matrici e dei vettori in $\mathbb{R}^N$, lavorando con il sistema lineare:

$$A_{Gal} \Psi = F_{Gal}, \quad \text{dove:}$$

$$A_{Gal}^{j,m} = \int_{\Gamma} \int_{\Gamma} \Phi_k(x, y) \phi_m(x) \overline{\phi_j(y)} ds(y) ds(x), \quad F_{Gal}^j = \int_{\Gamma} g_D(x) \overline{\phi_j(y)} ds(x). \quad (8)$$

In particolare, scegliendo ancora come funzioni ϕ_j le costanti sugli elementi K_j , otteniamo:

$$A_{Gal} \Psi = F_{Gal}, \quad \text{dove:}$$

$$A_{Gal}^{j,m} = \int_{K_j} \int_{K_m} \Phi_k(x, y) ds(y) ds(x), \quad F_{Gal}^j = \int_{K_j} g_D(x) ds(x). \quad (9)$$

Sappiamo che gli integrali delle funzioni di Hankel sono singolari intorno all'origine, quindi il termine A_{Gal}^{jj} dovrà essere trattato di conseguenza. In particolare, usando la parametrizzazione in (5) e le stime in (7), insieme ad alcuni suggerimenti delle dispense del corso, riusciamo a trovare le seguenti espressioni per A_{Gal}^{jm} e A_{Gal}^{jj} :

$$A_{Gal}^{jm} = \frac{i}{4} \int_0^{h_m} \int_0^{h_j} H_0^{(1)}(k|p_j + t\tau_j - p_m - s\tau_m|) ds dt$$

$$A_{Gal}^{jj} = \frac{ih_j^2}{2} \int_0^1 (1 - \zeta) H_0^{(1)}(kh_j\zeta) d\zeta.$$

La costruzione della mesh è analoga, quindi basta creare una nuova funzione `build_galerkin` e usarla all'interno di `build_2D_solution_galerkin`, che non riportiamo poichè cambia una sola riga di codice.

```

1  function [A, F] = build_galerkin(V, h_ideale, k, dato_bordo,
2      q_quadratura)
3  %% V = vertici del poligono
4  % h_ideale = taglia (circa) della mesh
5  % k = k della eq di helmholtz
6  % dato_bordo = dato al bordo
7
8  [Nodes, Midpoints, Hj, Dirs] = create_nodes(V, h_ideale);
9
10 N = length(Nodes);
11 A = zeros(N);
12 [nodi_quadratura, w] = gaussquad(q_quadratura);
13 F = zeros(N, 1);
14 for j=1:N
15     xj = Midpoints(j);
16     for m=1:N
17         if j ~= m
18             hm = Hj(m);
19             pm = Nodes(m);
20             tau_m = Dirs(m);
21             hj = Hj(j);
22             pj = Nodes(j);
23             tau_j = Dirs(j);
24             A(j, m) = 0.25i*hm*sum(w'.*( hj*sum( w.*besselh(0,
25                 k*abs(pm + ( hm*nodi_quadratura )*tau_m -
26                 pj - ( hj*nodi_quadratura' )*tau_j ))));
27         else
28             hj = Hj(j);
29             tau_j = Dirs(j);
30             A(j, j) = 1i*hj^2/2 * sum(w.*(1-nodi_quadratura).*
31                 besselh(0, k*hj*nodi_quadratura) );
32         end
33     end
34 end
35 F(j) = hj*sum(w.*dato_bordo ( (pj + ( hj*
36     nodi_quadratura )*tau_j)));
37 end
38 end

```

Il metodo di collocazione e quello di Galerkin danno risultati qualitativi sostanzialmente identici, come si può vedere dalla figura 6. Possiamo studiare quanto varia il tempo di risoluzione per i due metodi.

Una simulazione in cui viene risolto 10 volte diversi problemi (tutti con $k = 20$), su un computer con processore Intel(R) Core(TM) Ultra 9 285H (2.90 GHz), RAM installata 32.0 GB e scheda grafica Intel(R) Arc(TM) 140T GPU (16GB), rileva che il tempo medio di risoluzione è:

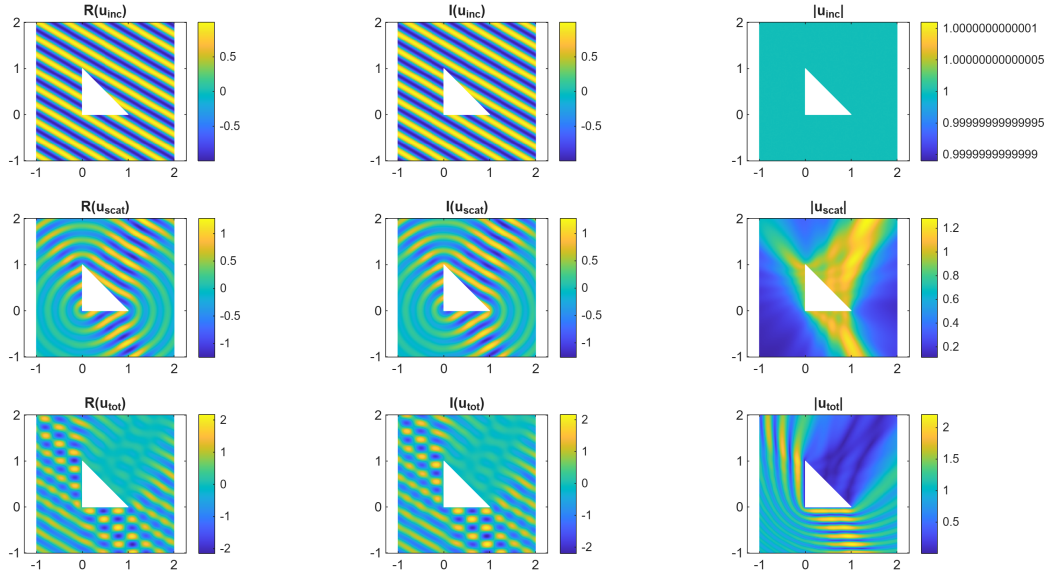


Figure 6: Metodo di Galerkin applicato al primo esempio

Scatterer	u_{inc}	collocazione	Galerkin
triangolo	onda piana	20.5132	29.2408
disco	onda piana	19.4606	27.8328
due triangoli	soluzione fondamentale centrata in x_0	21.2229	28.8246

Possiamo concludere quindi che dei due metodi, il metodo di collocazione è più veloce, ma entrambi hanno risultati ottimi. Abbiamo quindi costruito una routine di codici che ci permette di risolvere moltissime varianti dei problemi di scattering.

Tra i possibili sviluppi di questo progetto, si potrebbero implementare altri metodi come per esempio un metodo spettrale, oppure studiare ulteriormente come questi metodi numerici si comportano al variare dei parametri. Infatti quasi tutti i parametri in questo studio, come `h_ideale`, `q_quadatura...`, sono già stati selezionati in modo da portare risultati ottimali.

Bibliography

- [NIS] NIST DLMF. *NIST Digital Library of Mathematical Functions*. <https://dlmf.nist.gov/>, Release 1.2.8 of 2026-09-15. F. W. J. Olver, A. B. Olde Daalhuis, D. W. Lozier, B. I. Schneider, R. F. Boisvert, C. W. Clark, B. R. Miller, B. V. Saunders, H. S. Cohl, and M. A. McClain, eds. URL: <https://dlmf.nist.gov/>.